U3: Kompozicija (has-a ryšys)

Savaitės: 5-6
Svoris: 1 balas
Terminas: Savaitės 6 pabaiga

👉 Žr. **Užduočių Gidas**

🎯 Užduoties tikslas

Išmokti kurti sudėtingesnes klases su **kompozicija** (has-a ryšys).

📚 Mokymosi tikslai

Atlikę šią užduotį, mokėsite:

- ☑ Kurti klases, kurios "turi"/talpina kitas klases (kompozicija)
- ☑ Dirbti su rodyklėmis į objektus (`Objektas*`)
- ☑ Dirbti su `vector<Objektas>` (objektų kolekcija)
- ☑ Naudoti **statinius narius** (klasės lygio duomenys)
- ☑ Kurti **globalias funkcijas** (ne metodus)
- ☑ Suprasti kompozicijos vs agregacijos skirtumus

💡 Kas yra kompozicija?

Kompozicija - tai struktūrinis ryšys/santykis, kai vienas objektas **turi/patalpina savyje** (contains) kitą objektą.

Pavyzdys:

- **Kambarys** HAS **Langą** (has-a)
- **Kambarys** HAS **Duris** (has-a)

Skirtumas nuo paveldėjimo:

- **Paveldėjimas**: `Automobilis` IS-A `TransportoPriemone` (vėlesnė tema)
- **Kompozicija**: `Automobilis` HAS-A `Variklis` (ši tema)

Šios užduoties kompozicinė objektų struktūra:

```
Kambarys (konteineris/talpykla)
  ├─ Langai (komponentai - naudojama REIKŠMĖS semantika/prasmė)
  │   ├─ Langas 1
  │   ├─ Langas 2
  │   └─ ...
```

```
    └─ Durys (komponentai - naudojama RODYKLĖS semantika)
        └─ Durys*
```

```
Room (container)
  ├─ Windows (components - value semantics)
  │   ├─ Window 1
  │   ├─ Window 2
  │   └─ ...
  └─ Doors (component - pointer semantics)
      └─ Door*
```

📋 Užduoties žingsniai

1 žingsnis: Langas ir Durys klasės

Direktorija: `U3/01/`

Reikalavimai:

- ☑ Sukurkite **2 paprastas klases** (panaudokite U2 įgūdžius)

Klasė `Langas`:

`private` atributai/savybės (duomenys-nariai):

- `float aukstis` - lango aukštis (metrais)
- `float plotis` - lango plotis (metrais)
- `static int sukurtaLangu` - **statinis** skaitiklis (kiek langų sukurta)

`public` metodai (funkcijos-nariai):

- `Langas(float a, float p)` - konstruktorius (parametrinis)
  - Inicializuoja `aukstis`, `plotis`
  - Padidina `sukurtaLangu++`
  - Logging: `[DEBUG] Langas sukurtas...`
- `~Langas()` - destruktorius
  - Sumažina `sukurtaLangu--`
  - Logging: `[DEBUG] Langas sunaikintas...`
- `float gautiAuksti() const` - getter'is
- `float gautiPloti() const` - getter'is
- `float apskaiciuotiPlota() const` - aukštis × plotis
- `static int gautiSukurtaLangu()` - klasės lygio getter (statinis)

Klasė `Durys`:

Analogiškai kaip `Langas`:

- `private`: `aukstis`, `plotis`, `static int sukurtaDuru`

- `public`: konstruktorius, destruktorius, atributų getter'iai, `apskaiciuotiPlota()`, statinis skaitliuko getter'is

Svarbu!

Kodėl nėra default konstruktoriaus?

- Langas ir Durys **visuomet** turi turėti matmenis
- Default konstruktorius (`Langas()`) sukurtų "nelogiškus" objektus (0×0)
- **Geroji praktika**: jei objektui būtina pradinė būsena — parametrinis konstruktorius

> **Bandymas:**
>
> ```
> Langas langas;  // ❌ KLAIDA: no matching constructor
> ```

Tai **sąmoningas** klasės suprojektavimas — langai be matmenų neegzistuoja!

Testavimas:

main.cpp:

```
#include <iostream>
#include "Langas.h"
#include "Durys.h"
using namespace std;

int main() {
    cout << "Pradžioje langų: " << Langas::gautiSukurtaLangu() << endl;

    // Sukuriame 2 langus
    Langas l1(1.5, 1.0);
    Langas l2(2.0, 1.2);

    cout << "Lango 1 plotas: " << l1.apskaiciuotiPlota() << " m²" << endl;
    cout << "Dabar langų: " << Langas::gautiSukurtaLangu() << endl;

    // Sukuriame 2 duris
    Durys d1(2.0, 0.9);
    Durys d2(2.1, 0.8);

    cout << "Durų 1 plotas: " << d1.apskaiciuotiPlota() << " m²" << endl;
    cout << "Dabar durų: " << Durys::gautiSukurtaDuru() << endl;

    return 0;
}
```

Tikėtina išvestis:

```
Pradžioje langų: 0
[DEBUG] Langas sukurtas (1.5×1). Viso: 1
[DEBUG] Langas sukurtas (2×1.2). Viso: 2
Lango 1 plotas: 1.5 m²
Dabar langų: 2
[DEBUG] Durys sukurtos (2×0.9). Viso: 1
[DEBUG] Durys sukurtos (2.1×0.8). Viso: 2
Durų 1 plotas: 1.8 m²
Dabar durų: 2
[DEBUG] Durys sunaikintos. Liko: 1
[DEBUG] Durys sunaikintos. Liko: 0
[DEBUG] Langas sunaikintas. Liko: 1
[DEBUG] Langas sunaikintas. Liko: 0
```

Technikos:

- Statiniai nariai **inicijuojami už klasės ribų** (`.cpp` faile):

```
int Langas::sukurtaLangu = 0;
```

- Jei metodų apibrėžimai ne klasėje, t.y. *out-of-line* (šioje užduotyje `.cpp` failuose), naudojame klasės konteksto (*scope resolution*) operaciją `Klases_vardas::`

Failų struktūra:

```
U3/01/
├── Langas.h
├── Langas.cpp
├── Durys.h
├── Durys.cpp
├── main.cpp
└── Makefile
```

---

2 žingsnis: Refaktoringas į tarptautines praktikas

Direktorija: `U3/02/`

Motyvacija:

Lietuviški pavadinimai geri mokantis, bet **realiuose projektuose**:

- Anglų kalba — standartas
- Skirtingos komandos naudoja skirtingas vardų konvencijas
- Reikia susipažinti su abiem praktikomis

Reikalavimai:

Pakeiskite **Langas** ir **Durys** klases panaudojant:

- **Angliškus** failų, klasių vardus (`Window`, `Door`)
- **Angliškus** duomenų ir metodų (bent jau *getter/setter*) pavadinimus
- Skirtingas narių užvadinimo (*member naming*) praktikas

A. Window klasė — `this->` praktika

Window.h:

```
#ifndef WINDOW_H
#define WINDOW_H

class Window {
private:
    float height;
    float width;
    static int windowCount;

public:
    Window(float h, float w);
    ~Window();

    // UŽDUOTIS: Sukurti getters
    float getHeight() const;
    float getWidth() const;
    float calculateArea() const;

    static int getWindowCount();
};

#endif
```

Window.cpp — PAVYZDYS su `this->`:

```cpp
#include "Window.h"
#include <iostream>

int Window::windowCount = 0;

// PAVYZDYS: konstruktorius su this->
Window::Window(float height, float width) {
    // Parametrai turi TĄ PATĮ vardą kaip member'iai
    this->height = height;  // this-> nurodo į member
    this->width = width;

    windowCount++;
    std::cout << "[DEBUG] Window created (" << this->height
              << "x" << this->width << "). Total: "
              << windowCount << std::endl;
}

Window::~Window() {
    windowCount--;
    std::cout << "[DEBUG] Window destroyed. Remaining: "
              << windowCount << std::endl;
}

// JŪSŲ KODAS - getters:
float Window::getHeight() const {
    // UŽDUOTIS: grąžinti height
}

float Window::getWidth() const {
    // UŽDUOTIS: grąžinti width
}

float Window::calculateArea() const {
    // UŽDUOTIS: grąžinti height * width
}

int Window::getWindowCount() {
    return windowCount;
}
```

B. Door klasė — _ prefix praktika

Door.h:

```cpp
#ifndef DOOR_H
#define DOOR_H

class Door {
private:
```

```
    float _height;  // Underscore prefix!
    float _width;
    static int _doorCount;

public:
    Door(float h, float w);
    ~Door();

    // UŽDUOTIS: Sukurti getters (tie patys kaip Window)
    float getHeight() const;
    float getWidth() const;
    float calculateArea() const;

    static int getDoorCount();
};

#endif
```

Door.cpp — JŪSŲ KODAS:

```
#include "Door.h"
#include <iostream>

int Door::_doorCount = 0;

Door::Door(float h, float w) {
    // UŽDUOTIS: priskirti _height = h, _width = w
    // UŽDUOTIS: padidinti _doorCount
    // UŽDUOTIS: išspausdinti debug žinutę
}

Door::~Door() {
    // UŽDUOTIS: sumažinti _doorCount
    // UŽDUOTIS: išspausdinti debug žinutę
}

// UŽDUOTIS: Implementuoti getterius (panašiai kaip Window)
float Door::getHeight() const {
    // UŽDUOTIS: grąžinti _height
}

float Door::getWidth() const {
    // UŽDUOTIS: grąžinti _width
}

float Door::calculateArea() const {
    // UŽDUOTIS: grąžinti _height * _width
}

int Door::getDoorCount() {
    return _doorCount;
}
```

Testavimas:

main.cpp:

```
#include <iostream>
#include "Window.h"
#include "Door.h"
using namespace std;

int main() {
    cout << "=== Enkapsuliacijos demonstracija ===" << endl;

    Window window(1.5, 1.0);
    Door door(2.0, 0.9);

    // TEISINGAI - public interfeisas:
    cout << "Window height: " << window.getHeight() << endl;
    cout << "Door area: " << door.calculateArea() << endl;

    // NETEISINGAI - bandymas pasiekti private (užkomentuota):
    // cout << window.height;   // ❌ KLAIDA!

    cout << "\n=== new operatorius ===" << endl;

    Door* pDoor = new Door(2.5, 1.0);  // ← Dinaminis objektas

    cout << "Dynamic door: " << pDoor->getHeight()
         << "×" << pDoor->getWidth() << endl;

    delete pDoor;  // Atlaisviname atmintį

    cout << "\n=== nullptr demonstracija ===" << endl;

    Door* pDoor2 = nullptr;

    if (pDoor2 != nullptr) {
        cout << "Door area: " << pDoor2->calculateArea() << endl;
    } else {
        cout << "Door doesn't exist (nullptr)" << endl;
    }
    // Kodėl nullptr svarbu? Pabandykite atkomentuoti:
    // pDoor2->calculateArea();  // ← Segmentation fault!

    return 0;
}
```

Testinė išvestis:

```
=== Enkapsuliacijos demonstracija ===
[DEBUG] Window created (1.5×1). Total: 1
[DEBUG] Door created (2×0.9). Total: 1
Window height: 1.5
Door area: 1.8

=== new operatorius ===
[DEBUG] Door created (2.5×1). Total: 2
Dynamic door: 2.5×1
[DEBUG] Door destroyed. Remaining: 1

=== nullptr demonstracija ===
Door doesn't exist (nullptr)
[DEBUG] Door destroyed. Remaining: 0
[DEBUG] Window destroyed. Remaining: 0
```

Technikos:

Naming conventions:

- `getX()` / `setX()` — Java/C# stilius (populiariausias)
- `get_x()` / `set_x()` — Python/snake_case (rečiau C++)
- Lietuviški (`gautiX()`) — mokymuisi, ne produkcijai

Member vs parametrų vardai:

1. **`this->` naudojimas** (Window):
   - Parametras ir member'is turi **tą patį** vardą
   - `this->height = height;` — aiškiai atskiria
   - Privalumas: natūralūs vardai
2. **Underscore prefix** (Door):
   - Member'iai: `_height`, parametrai: `h` arba `height`
   - Nereikia `this->`
   - Privalumas: aiškus skirtumas kode
3. **Skirtingi vardai**:
   - Member: `height`, parametras: `h`
   - Paprasčiausias, bet ne visada aiškus

🆕 **NAUJAS OPERATORIUS — `new` / `delete`:**

- `new Type` — sukuria objektą **dinamiškai** (heap'e)
- `delete pointer` — atlaisvina atmintį (BŪTINA!)
- **Skirtumas nuo stack**:
  - Stack: `Door door(2.0, 0.9);` — automatiškai sunaikinama
  - Heap: `Door* p = new Door(2.0, 0.9);` — **reikia delete!**

- **Memory leak** — jei pamiršite `delete`

`nullptr` **(C++11+):**

- ☑ `nullptr` — type-safe, modernas
- ⚠ `NULL` — C makro (legacy)
- ✕ `0` — integer, ne pointer

Inkapsuliacijos principas = Informacijos slėpimas (dažna interpretacija):

- **Private** nariai **nepasiekiami** iš išorės
- **Privaloma** naudoti **public** interfeisą (getters/setters)
- Bandymas `window.height` → kompiliavimo klaida
- Tai **apsaugo** duomenis ir leidžia **validaciją**

Inkapsuliacijos ir informacijos slėpimo skirtumas (tradicinė interpretacija):

- **Inkapsuliacijos (encapsulation)** — duomenų ir metodų sujungimas į vieną vienetą (klasę)
- **Informacijos slėpimas (information hiding)** — private/public mechanizmas

Failų struktūra:

```
U3/02/
├── Window.h, Window.cpp
├── Door.h, Door.cpp
├── main.cpp
└── Makefile
```

---

3 žingsnis: Room klasė (kompozicija)

Direktorija: `U3/03/`

Motyvacija:

Iki šiol turėjome **paprastas** klases (`Window`, `Door`).

Dabar kursime **sudėtingesnę** klasę, kuri naudoja du būdus "turėti"/"susidėti" informaciją/duomenis apie objektus:

1. patalpindami (klasės išorėje sukurtų) objektų **kopijas**
2. saugodami **rodykles** į (klasės išorėje sukurtus) objektus

Realaus pasaulio analogija: Kambarys "turi" langus ir duris — tai natūrali **has-a** struktūra!

Svarbu!

Prieš pradedant šį žingsnį, **užkomentuokite** `Window` ir `Door` klasių konstruktorių/destruktorių [DEBUG] logging išvestis:

```
Window::Window(float height, float width) {
    this->height = height;
    this->width = width;
    windowCount++;
    // std::cout << "[DEBUG] Window created..." << std::endl;  ← Užkomentuota
}

Window::~Window() {
    windowCount--;
    // std::cout << "[DEBUG] Window destroyed..." << std::endl;  ← Užkomentuota
}
```

Kodėl?

- Dabar susikoncentruojame į **Room kompoziciją**
- "Virtuvė" (objektų kopijavimas, *copy semantics*) bus atskleista vėliau (bonus žingsnyje)
- Naudojame **natūralius C++ primityvus** kompoziciją pademonstruoti be [DEBUG] "triukšmo"

Reikalavimai:

Sukurkite **Room** klasę, kuri susideda iš:

Privatūs nariai:

- `float length` — kambario ilgis (m)
- `float width` — kambario plotis (m)
- `float height` — kambario aukštis (m)
- `vector<Window> windows` — **langų kolekcija** (objektų kopijos)

- `Door* door` — **rodyklė į duris** (saugome tik adresą)

Vieši metodai:

Konstruktorius:

- `Room(float l, float w, float h)`
  - Inicializuoja matmenis (atributus)
  - Inicializuoja `door = nullptr` (kol kas durų nėra)
  - Logging: `[DEBUG] Room created...`

Destruktorius:

- `~Room()`
  - Logging: `[DEBUG] Room destroyed`
  - **Pastaba:** `vector` automatiškai sunaikina Window objektus
  - Door **nenaikinome** — ji išorėje (main.cpp atsakingas)

Kompozicijos metodai:

- `void addWindow(const Window& w)`
  - Prideda langą į `windows` kolekciją
  - **Kas nutinka?** `vector` sukuria Window **kopiją** ir laiko ją savyje
  - **Kodėl `const Window&`?** Efektyvumas — objektą perduodame pagal nuoodą, ne pagal reikšmę (SVARBU!)
  - Logging: `[INFO] Window added. Total: X`
- `void setDoor(Door* d)`
  - Nustato durų rodyklę
  - **Kas nutinka?** Room laiko tik **adresą**, ne patį objektą
  - Door objektas yra sukurtas su `new` **išorėje** (`main.cpp`)

Getter'iai:

- `float getLength() const` — kambario ilgis
- `float getWidth() const` — kambario plotis
- `float getHeight() const` — kambario aukštis
- `int getNumberOfWindows() const` — langų skaičius (`windows.size()`)
- `const Door* getDoor() const` — durų rodyklė (read-only)

Room.h:

```
#ifndef ROOM_H
#define ROOM_H

#include <vector>
#include "Window.h"
#include "Door.h"
```

```
class Room {
private:
    float length;
    float width;
    float height;

    std::vector<Window> windows;  // Langų vektorius
    Door* door;                    // Tik rodyklė

public:
    Room(float l, float w, float h);
    ~Room();

    // Kompozicijos metodai
    void addWindow(const Window& w);
    void setDoor(Door* d);

    // Getters
    float getLength() const;
    float getWidth() const;
    float getHeight() const;
    int getNumberOfWindows() const;
    const Door* getDoor() const;
};

#endif
```

Room.cpp — JŪSŲ KODAS:

```
#include "Room.h"
#include <iostream>

Room::Room(float l, float w, float h)
    : length(l), width(w), height(h), door(nullptr)
{
    std::cout << "[DEBUG] Room created (" << length
              << "x" << width << "x" << height << ")" << std::endl;
}

Room::~Room() {
    std::cout << "[DEBUG] Room destroyed" << std::endl;
    // UŽDUOTIS: Ar reikia delete windows? (Ne! vector automatiškai)
    // UŽDUOTIS: Ar reikia delete door? (Ne! main.cpp atsakingas)
}

void Room::addWindow(const Window& w) {
    // UŽDUOTIS: pridėti langą į windows vector
    // windows.push_back(w);

    // UŽDUOTIS: išspausdinti "[INFO] Window added. Total: X"
```

```
    // std::cout << "[INFO] Window added. Total: " << windows.size() << std::endl;
}

void Room::setDoor(Door* d) {
    // UŽDUOTIS: priskirti door = d
}

// UŽDUOTIS: Implementuoti visus getterius

float Room::getLength() const {
    // UŽDUOTIS: grąžinti length
}

float Room::getWidth() const {
    // UŽDUOTIS: grąžinti width
}

float Room::getHeight() const {
    // UŽDUOTIS: grąžinti height
}

int Room::getNumberOfWindows() const {
    // UŽDUOTIS: grąžinti windows.size()
}

const Door* Room::getDoor() const {
    // UŽDUOTIS: grąžinti door
}
```

Testavimas:

main.cpp: pavyzdys - bandykite, modifikuokite palei save

```
#include <iostream>
#include "Window.h"
#include "Door.h"
#include "Room.h"
using namespace std;

int main() {
    cout << "=== Kompozicijos demonstracija ===" << endl;

    Room room(5.0, 4.0, 3.0);

    cout << "\n--- Pridedame langus ---" << endl;
    Window w1(1.5, 1.0);
    Window w2(1.5, 1.0);

    cout << "Window 1 area: " << w1.calculateArea() << " m²" << endl;
    cout << "Window 2 area: " << w2.calculateArea() << " m²" << endl;
```

```
    room.addWindow(w1);  // Kopijuojasi
    room.addWindow(w2);

    cout << "\n--- Pridedame duris ---" << endl;
    Door* door = new Door(2.0, 0.9);
    cout << "Door area: " << door->calculateArea() << " m²" << endl;
    room.setDoor(door);  // Tik rodyklė

    cout << "\n--- Kambario informacija ---" << endl;
    cout << "Room dimensions: " << room.getLength()
         << "×" << room.getWidth()
         << "×" << room.getHeight() << " m" << endl;
    cout << "Windows: " << room.getNumberOfWindows() << endl;

    if (room.getDoor() != nullptr) {
        cout << "Door: " << room.getDoor()->getHeight()
             << "×" << room.getDoor()->getWidth() << " m" << endl;
    }

    cout << "\n--- Atlaisviname atmintį ---" << endl;
    delete door;  // main.cpp atsakingas!

    cout << "\n--- Programa baigiasi (Room sunaikinamas) ---" << endl;
    return 0;
}
```

Testinė išvestis:

```
=== Kompozicijos demonstracija ===
[DEBUG] Room created (5×4×3)

--- Pridedame langus ---
Window 1 area: 1.5 m²
Window 2 area: 1.5 m²
[INFO] Window added. Total: 1
[INFO] Window added. Total: 2

--- Pridedame duris ---
Door area: 1.8 m²

--- Kambario informacija ---
Room dimensions: 5×4×3 m
Windows: 2
Door: 2×0.9 m

--- Atlaisviname atmintį ---

--- Programa baigiasi (Room sunaikinamas) ---
[DEBUG] Room destroyed
```

Technikos:

Du būdai saugoti objektus:

1. **Kopija** (`vector<Window> windows`):
   - `addWindow(w1)` — **nukopijuoja** visą Window objektą
   - Room **laiko** Window kopijas savyje
   - Destruktoriuje `vector` **automatiškai** sunaikina langus
   - **Analogija:** Kambarys turi savo langus
2. **Rodyklė** (`Door* door`):
   - `setDoor(d)` — **laiko tik adresą**
   - Door objektas yra **išorėje** (main.cpp sukūrė su `new`)
   - Room destruktorius **NENAIKINA** durų
   - **main.cpp atsakingas** už `delete door`
   - **Analogija:** Kambarys žino, kur yra durys, bet jos ne jo nuosavybė

Kompozicijos atsakomybės (DUK):

❓ **Ar Room turi duris?**

- Ne! Room tik **žino apie** duris (laiko rodyklę)
- Durys **priklauso** main.cpp (kas sukūrė su `new`)

❓ **Kas atlaisvina `door` atmintį?**

- **main.cpp** daro `delete door` (kas sukūrė, tas ir naikina)
- Room destruktorius jų **neliečia**

❓ **Kas atsitinka su `windows`?**

- `vector` **automatiškai** sunaikina visus Window objektus
- Room destruktoriuje nieko papildomo daryti nereikia

Kompozicija vs Agregacija:

Šiame žingsnyje matėme ABU:

Kompozicija (`vector<Window> windows`):

- Room **valdo** Window gyvavimo ciklą
- Window **negali** egzistuoti be Room
- Room sunaikinus → automatiškai sunaikinami ir Window
- **"has-a" (strong ownership)**

Agregacija (`Door* door`):

- Room **žino apie** Door, bet **nevaldo**
- Door **gali** egzistuoti nepriklausomai
- Room sunaikinus → Door lieka (main.cpp valdo)

- **"knows-about" (weak relationship)**

Kada ką naudoti?

- Kompozicija: komponentas **priklauso** tik šiam konteineriui
- Agregacija: komponentas **dalinamas** ar egzistuoja nepriklausomai

Pavyzdys:

- Automobilis **turi** variklį (kompozicija) — variklis nepriklauso be auto
- Automobilis **turi** vairuotoją (agregacija) — vairuotojas gali egzistuoti atskirai

Failų struktūra:

```
U3/03/
├── Window.h, Window.cpp
├── Door.h, Door.cpp
├── Room.h, Room.cpp
├── main.cpp
└── Makefile
```

---

4 žingsnis: Globalios funkcijos (skaičiavimai)

Direktorija: `U3/04/`

Motyvacija:

Kambariui reikia **skaičiavimų** (sienų plotas, grindjuosčių ilgis). Ar šie metodai turėtų būti **Room klasėje**?

Ne! Tai **atskirti funkcionalumai** → **globalios funkcijos** (ne metodai).

Kodėl globalios?

- **Separation of concerns** — `Room` neturi "žinoti" apie statybos skaičiavimus
- Galima **skirtingi algoritmai** (pvz., su/be angokraščių)
- **Testabilumas** — lengviau testuoti atskirai

Reikalavimai:

Sukurkite **papildomą modulį** `calculations.h` + `calculations.cpp` su dviem funkcijomis:

1. Sienų ploto skaičiavimas:

```
float calculateWallArea(const Room& room);
```

Algoritmas:

- Perimetras = 2 × (ilgis + plotis)
- Sienų plotas = perimetras × aukštis
- Atimti visų langų plotus
- Atimti durų plotą

2. Grindjuosčių ilgio skaičiavimas:

```
float calculateBaseboardLength(const Room& room);
```

Algoritmas:

- Perimetras = 2 × (ilgis + plotis)
- Atimti durų plotį

---

calculations.h:

```
#ifndef CALCULATIONS_H
#define CALCULATIONS_H

#include "Room.h"
```

```
// Globalios funkcijos skaičiavimams
float calculateWallArea(const Room& room);
float calculateBaseboardLength(const Room& room);

#endif
```

Problema:

Bandymas (su klaida):

```
float calculateWallArea(const Room& room) {
    // ...
        for (int i = 0; i < room.getNumberOfWindows(); i++) {
        wallArea -= room.windows[i].calculateArea();  // ❌ KLAIDA!
    }
    // ...
}
```

Kompiliavimo klaida:

```
error: 'std::vector<Window> Room::windows' is private within this context
     wallArea -= room.windows[i].calculateArea();
                      ^~~~~~~
Room.h:14:23: note: declared private here
     std::vector<Window> windows;
                         ^~~~~~~
```

Kodėl?

- `windows` yra **private** `Room` narys
- Globali funkcija **negali** jo pasiekti
- **Inkapsuliacijos principas** — privaloma naudoti public interfeisą!

Sprendimas:

Room.h (papildyti):

```
class Room {
    // ... esami nariai

public:
    // ... esami metodai
```

```
    // Pridėti: gauti individualų langą
    const Window& getWindow(int index) const;
};
```

Room.cpp (papildyti):

```
const Window& Room::getWindow(int index) const {
    return windows[index];
}
```

---

calculations.cpp — JŪSŲ KODAS:

```
#include "calculations.h"
#include <iostream>

float calculateWallArea(const Room& room) {
    // UŽDUOTIS: Apskaičiuoti perimetrą
    // float perimeter = 2 * (room.getLength() + room.getWidth());

    // UŽDUOTIS: Apskaičiuoti sienų plotą
    // float wallArea = perimeter * room.getHeight();

    // UŽDUOTIS: Atimti langų plotus
    // for (int i = 0; i < room.getNumberOfWindows(); i++) {
    //     const Window& window = room.getWindow(i);  // Per getter!
    //     wallArea -= window.calculateArea();
    // }

    // UŽDUOTIS: Atimti durų plotą
    // const Door* door = room.getDoor();
    // if (door != nullptr) {
    //     wallArea -= door->calculateArea();
    // }

    // return wallArea;
}

float calculateBaseboardLength(const Room& room) {
    // UŽDUOTIS: Apskaičiuoti perimetrą
    // float perimeter = 2 * (room.getLength() + room.getWidth());

    // UŽDUOTIS: Atimti durų plotį
    // const Door* door = room.getDoor();
    // if (door != nullptr) {
    //     perimeter -= door->getWidth();
    // }
```

```
    // return perimeter;
}
```

Testavimas:

main.cpp: pavyzdys - bandykite, modifikuokite palei save

```
#include <iostream>
#include "Window.h"
#include "Door.h"
#include "Room.h"
#include "calculations.h"
using namespace std;

int main() {
    cout << "=== Kambario skaičiavimai ===" << endl;

    Room room(5.0, 4.0, 3.0);

    cout << "\n--- Pridedame langus ---" << endl;
    Window w1(1.5, 1.0);
    Window w2(1.5, 1.0);

    cout << "Window 1 area: " << w1.calculateArea() << " m²" << endl;
    cout << "Window 2 area: " << w2.calculateArea() << " m²" << endl;

    room.addWindow(w1);
    room.addWindow(w2);

    cout << "\n--- Pridedame duris ---" << endl;
    Door* door = new Door(2.0, 0.9);
    cout << "Door area: " << door->calculateArea() << " m²" << endl;
    room.setDoor(door);

    cout << "\n--- Kambario informacija ---" << endl;
    cout << "Room dimensions: " << room.getLength()
         << "×" << room.getWidth()
         << "×" << room.getHeight() << " m" << endl;
    cout << "Windows: " << room.getNumberOfWindows() << endl;

    if (room.getDoor() != nullptr) {
        cout << "Door: " << room.getDoor()->getHeight()
             << "×" << room.getDoor()->getWidth() << " m" << endl;
    }

    cout << "\n--- Skaičiavimai (globalios funkcijos) ---" << endl;
    float wallArea = calculateWallArea(room);
    float baseboardLength = calculateBaseboardLength(room);

    cout << "Wall area (minus windows/door): "
```

```
            << wallArea << " m²" << endl;
    cout << "Baseboard length (minus door): "
         << baseboardLength << " m" << endl;

    cout << "\n--- Atlaisviname atmintį ---" << endl;
    delete door;

    cout << "\n--- Programa baigiasi ---" << endl;
    return 0;
}
```

Testinė išvestis:

```
=== Kambario skaičiavimai ===
[DEBUG] Room created (5×4×3)

--- Pridedame langus ---
Window 1 area: 1.5 m²
Window 2 area: 1.5 m²
[INFO] Window added. Total: 1
[INFO] Window added. Total: 2

--- Pridedame duris ---
Door area: 1.8 m²

--- Kambario informacija ---
Room dimensions: 5×4×3 m
Windows: 2
Door: 2×0.9 m

--- Skaičiavimai (globalios funkcijos) ---
Wall area (minus windows/door): 49.2 m²
Baseboard length (minus door): 17.1 m

--- Atlaisviname atmintį ---

--- Programa baigiasi ---
[DEBUG] Room destroyed
```

Technikos:

Globalios funkcijos vs metodai:

- **Metodas** — priklauso klasei (`room.calculate()`)
- **Globali** — nepriskiriama klasei (`calculate(room)`)
- Kada naudoti globalias? Kai funkcionalumas **ne klasės atsakomybė**

const reference parametras:

- `const Room& room` — efektyvumas (nekopijuoja objekto)
- `const` — garantuoja, kad funkcija **nekeičia** objekto

Inkapsuliacijos principas:

- Globalios funkcijos **negali** pasiekti `private`
- **Privaloma** naudoti `public` interfeisą (getters)
- Jei reikia daugiau duomenų → pridėti getter

Separation of concerns:

- `Room` **saugo** duomenis
- Globalios funkcijos **skaičiuoja** pagal tuos duomenis
- Kiekvienas turi **savo atsakomybę**

Failų struktūra:

```
U3/04/
├── Window.h, Window.cpp
├── Door.h, Door.cpp
├── Room.h, Room.cpp
├── calculations.h, calculations.cpp  ← NAUJAS
├── main.cpp
└── Makefile
```

---

5 žingsnis: Planuojamas "bonus" žingsnis

```
Pateiktas atskirame faile (2026-03-19)
```

GitLab direktorijų struktūra:

```
└── U3/
    ├── README.md          ← Užduoties santrauka (PRIVALOMA)
    ├── 01/                ← Langas + Durys (lietuviški)
    ├── 02/                ← Window + Door (refaktoringas)
    ├── 03/                ← Room kompozicija
    └── 04/                ← Globalios funkcijos
```

Moodle pateikimas:

Pateikite:

- PDF su README.md turiniu
- GitLab repository nuorodą

README.md šablonas:

```
# U3: Kompozicija

**Studentas:** Vardas Pavardė
**Grupė:** X

## Žingsnių santrauka

| Žingsnis | Direktorija | Aprašymas |
|----------|-------------|-----------|
| 1 | `01/` | Langas ir Durys klasės (lietuviški) |
| 2 | `02/` | Refaktoringas (tarptautinės praktikos) |
| 3 | `03/` | Room kompozicija (vector + pointer) |
| 4 | `04/` | Skaičiavimai (globalios funkcijos) |

## Kompiliavimas
```bash
cd U3/04
make
./app
```

Pastabos

[Jūsų pastabos, sunkumai, įžvalgos...]
```

Git workflow:

```
git add U3/01/
git commit -m "U3: 1 žingsnis - Langas ir Durys klasės"
git push

git add U3/02/
git commit -m "U3: 2 žingsnis - Refaktoringas (tarptautinės praktikos)"
git push

git add U3/03/
git commit -m "U3: 3 žingsnis - Room kompozicija"
git push

git add U3/04/
git commit -m "U3: 4 žingsnis - Globalios funkcijos"
git push

git add U3/README.md
git commit -m "U3: README"
git push
```